# VASUDHAETHNIC

LUXURY INDIAN FASHION • SYSTEM ARCHITECTURE & IMPLEMENTATION REVIEW

| | |
|---|---|
| **Backend:** FastAPI (Python 3.13) + SQLAlchemy 2.0 | **Frontend:** React 18 + Vite + Tailwind CSS |
| **Database:** PostgreSQL 16+ with pgvector | **Security:** Bcrypt + JWT (Role-Based RBAC) |
| **AI Engine:** Local Zero-API-Key RAG + SentenceTransformers | **Status:** Production Ready (17/17 Tests Passed) |

## 1. Executive Summary

VasudhaEthnic is a full-stack, role-based luxury Indian ethnic fashion ecommerce platform. The application is built upon a 3-tier architecture: a React 18 frontend, a FastAPI REST API backend, and a PostgreSQL database with pgvector support. The platform features strict role-based access control (Customer vs. Seller), server-authoritative inventory with row-locking to guarantee zero overselling, and a privacy-first, local RAG shopping assistant operating without any external cloud API keys.

## 2. System Architecture & Component Flow

| Tier / Layer | Components & Responsibilities |
|---|---|
| **Frontend Client** (React 18 / Vite) | Luxury couture design (#0C3B10, #DBAB7D, #FFFFFF). Manages customer storefront, collections, bag, live checkout, customer order history, and Seller Management Suite with inventory matrix. |
| **API Gateway** (FastAPI / Uvicorn) | Asynchronous REST API at port 8000. Provides /api/auth, /api/products, /api/orders, /api/ai, and /health endpoints. Implements SlowAPI rate limiting, OWASP security headers, and MIME validation. |
| **Database Layer** (PostgreSQL + pgvector) | Relational schema with user accounts, products, variant stock matrix, and transactional orders. Uses with_for_update() row locks to prevent race conditions during checkout. |
| **Local AI Concierge** (Zero-API-Key RAG) | Local BAAI/bge-small-en-v1.5 embeddings running on CPU + hybrid vector search with SQL budget/size filtering + dynamic database inventory verification + local Ollama LLM. |

## 3. Security & Access Control Framework

- **Role-Based Access Control (RBAC):** Server-enforced dependency injection separates Customer capabilities (browsing, cart, checkout, AI assistant) from Seller actions (product creation/editing, variant stock updates, order fulfillment).
- **Password Security:** Passwords hashed using bcrypt with auto-salted key derivation.
- **Stateless JWT Authentication:** Signed tokens with configurable expiry (ACCESS_TOKEN_EXPIRE_MINUTES).
- **Prompt Injection Protection:** Pre-retrieval sanitizer strips jailbreak attempts and system prompt override patterns.
- **Security Headers:** Enforces X-Content-Type-Options (nosniff), X-Frame-Options (DENY), and X-XSS-Protection.
- **Safe Uploads:** File types restricted to JPEG, PNG, WEBP with 5MB cap and randomized UUID filenames.

## 4. Concurrency & Inventory Integrity

To ensure that high-demand luxury items (such as bridal lehengas with limited batch quantities) cannot be oversold, the backend uses atomic database transactions with row-level locks on the ProductVariant table:

```
variant = db.query(ProductVariant).filter(...).with_for_update().first()
```

If the requested quantity exceeds available stock, the transaction immediately rolls back and returns a descriptive HTTP 400 Out of Stock error before any order record is committed.

## 5. Verification & Test Suite Results

| Test Suite | Test Module Focus | Status |
|---|---|---|
| Auth Suite | Registration, Login, Password Validation, RBAC Enforcement | **PASSED** (5/5) |
| Inventory & Orders | Row-Lock Stock Deduction, Out-of-Stock Rejection, Atomic Commit | **PASSED** (2/2) |
| Product Management | Seller CRUD, Variant Matrix Stock Update, Public Listing | **PASSED** (3/3) |
| RAG AI Engine | Injection Detection, Constraint Extraction, RAG Payload Format | **PASSED** (4/4) |
| Security & System | Health Check DB Status, OWASP Headers, MIME Validation | **PASSED** (3/3) |
| **TOTAL** | **17 Automated Backend Test Suites** | **100% SUCCESS** |